

Predicting Final Chess Game Outcomes

Big Data Pipeline from Move-Level Board States

Ivan Chabanov, Alexander Mikhailov, Danil Eramasov, Bulat Fakhrutdinov

Big Data Course -- IU, 2026

Project Idea

Question: can we build a scalable system that predicts the final result of a chess game from intermediate board states?

The project uses Chess.com game archives and turns raw PGN records into a distributed analytical dataset.

The target task is:

```
board state + game context at a given ply  
-> white_win | black_win | draw
```

Business Motivation

Outcome prediction from board states supports:

- real-time win probability estimation
- training and coaching analytics
- opening and time-control analysis
- detection of decisive positional patterns
- scalable reporting for chess platforms

The Big Data focus is not only the model, but the complete path from raw data to warehouse, analytics, and dashboard.

Data Source

The dataset was generated from Chess.com public archive endpoints.

Each row represents **one ply**: one half-move from one game.

Level	Examples
Game metadata	players, ratings, result, time control, ECO opening
Move context	SAN/UCI move, side to move, move number
Board state	FEN, material, legal moves, phase flags
Derived chess features	pawn structure, king safety, castling, time pressure

Dataset Characteristics

Property	Value
Rows	674,594 plies
File size	589.22 MB
Feature count	100+ columns
Target	final_result_class
Classes	white_win, black_win, draw
Datetime fields	timestamp, UTC datetime, month, weekday, hour
Storage path	CSV -> PostgreSQL -> HDFS -> Hive

The dataset satisfies the course scale requirements.

Data Quality

The generated dataset is clean enough for distributed analysis.

Check	Result
API fetch failures	0
Malformed rows	0
Duplicate games	0
Duplicate (game_uuid, ply_index) rows	0
Skipped games	2

Known missingness is expected: Chess.com does not always expose clock annotations or accuracy metrics.

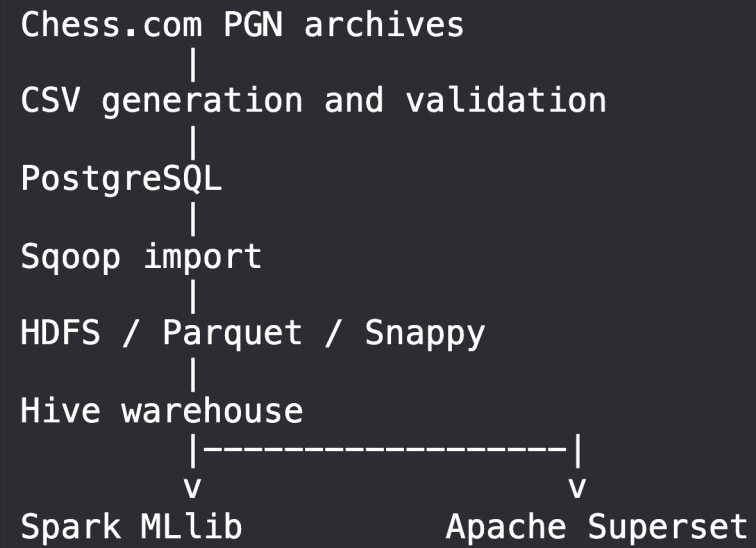
Result Distribution

Final outcome labels are available for every row.

Class	Rows
black_win	313,815
white_win	305,184
draw	55,595

Draws are the minority class, so weighted F1 should be reported together with accuracy in the ML section.

Architecture



Stage I: Ingestion

Stage I turns the local dataset into distributed storage.

Step	Implementation
Schema creation	Alembic migration
Local loading	PostgreSQL bulk insert script
Distributed import	Sqoop
HDFS format	Parquet with Snappy compression
Log artifact	output/postgres_results.txt

The raw warehouse table is created first, then optimized for analytical querying.

Schema Design

The project uses a denormalized `chess_moves` table.

Why this fits the project:

- every ply is directly usable for analytics and ML
- Spark can scan and vectorize without joining many tables
- game metadata and board-state features stay point-in-time
- indexes support common SQL analysis patterns

Trade-off: repeated game-level metadata increases storage size, but simplifies distributed processing.

Stage II: Hive Warehouse

The Hive table is optimized for repeated analytical queries.

Design choice	Value
Source table	chess_moves
Storage	Parquet
Compression	Snappy
Partition key	archive_month
Bucket key	game_uuid
Buckets	8

Partitioning supports archive-month filtering; bucketing supports game-level joins and grouped analysis.

Stage II: EDA Outputs

Eight HiveQL insights are produced by `scripts/stage2.sh`.

File	Question
q1.hql	outcome distribution by time class
q2.hql	top ECO openings by white win rate
q3.hql	outcome by rating-gap bucket
q4.hql	game length by time class
q5.hql	checkmate plies by game phase
q6.hql	outcome trends by archive month
q7.hql	outcome by termination type
q8.hql	material advantage and outcome

EDA Granularity

The dataset is ply-level, but not every insight should count plies.

Game-level insights use:

```
WHERE ply_index = 1
```

This prevents long games from being over-weighted.

Ply-level insights are used only where the unit of analysis is genuinely a move or board state.

Insight: Rating Gap

`q3.hql` groups games by:

```
rating_diff = white_rating - black_rating
```

Expected story:

- negative gap means Black was higher rated
- positive gap means White was higher rated
- win rates should move monotonically with rating advantage
- draw share gives context for balanced matchups

This insight supports fair matchmaking and Elo-style analysis.

Insight: Material Advantage

`q8.hql` selects one middlegame ply per game and buckets material balance.

Bucket examples	Meaning
<code>white_down>=5</code>	White is losing significant material
<code>equal</code>	Material balance is even
<code>white_up_2_4</code>	White has a moderate advantage
<code>white_up>=5</code>	White has a decisive material edge

This connects board-state features to final outcome.

Insight: Openings

`q2.hql` ranks ECO openings by white win percentage.

Noise is reduced by requiring at least 200 games per ECO code.

The analysis can be used to:

- identify high-impact openings
- compare opening popularity and success
- support training recommendations
- add opening filters to Superset

Dashboard Layer

Stage IV is presented through Apache Superset.

Each insight has:

- a Hive result table
- an exported CSV artifact
- a manually exported chart image
- a dashboard interpretation

The dashboard combines dataset scale, outcome distributions, rating effects, opening effects, material effects, and temporal trends.

Current Status

The ingestion, warehouse, and EDA story is presentation-ready. The ML section below is intentionally a template: final model metrics should be filled in only after the Spark MLlib part is implemented and evaluated.

ML Section Template

Reserved for the pending Spark MLlib work

ML Task Template

Problem type: multiclass classification.

Target:

```
final_result_class = white_win | black_win | draw
```

Prediction point: a selected ply-level board state.

Important boundary: do not use leakage columns such as `result_raw`, `termination`, `white_won_flag`, `black_won_flag`, or `draw_flag` as model features.

Feature Template

Candidate feature groups:

Group	Examples
Ratings	<code>white_rating</code> , <code>black_rating</code> , <code>rating_diff</code>
Time control	<code>time_class</code> , base seconds, increment
Board material	material diff, piece counts, non-pawn material
Pawn structure	doubled, isolated, passed pawns
King safety	pawn shield, king tropism, check flag
Phase	opening, middlegame, endgame, ply bucket

Spark Pipeline Template

```
Hive table
|
Spark DataFrame
|
train / test split by game_uuid
|
StringIndexer + OneHotEncoder
|
VectorAssembler
|
model training
|
cross-validation
|
test-set evaluation
```

Use a game-level split to avoid placing plies from the same game in both train and test.

Model Comparison Template

Model	Hyperparameters	Accuracy	Weighted F1
Random Forest	numTrees , maxDepth	TBD	TBD
SVM / One-vs-Rest	regParam , maxIter	TBD	TBD
Naive Bayes	smoothing	TBD	TBD

Add the best model only after cross-validation and held-out testing are complete.

Evaluation Template

Report at minimum:

- accuracy
- weighted F1
- per-class precision and recall
- confusion matrix
- baseline comparison
- notes on draw-class performance

Recommended baseline:

`predict the majority class on the training split`

This makes the model value measurable rather than assumed.

Prediction Example Template

Replace this slide after ML is ready.

Field	Value
Ply phase	TBD
Material balance	TBD
Rating gap	TBD
Time class	TBD
Predicted outcome	TBD
Model confidence	TBD

The example should come from the held-out test split.

Final Findings

What is already defensible:

1. The project builds a full Big Data path from Chess.com archives to Hive.
2. The dataset has 674,594 ply-level records and 100+ engineered features.
3. Hive storage uses Parquet, Snappy compression, partitioning, and bucketing.
4. Eight EDA queries cover time control, openings, rating gaps, material, phases, and trends.
5. Superset is the presentation layer for interactive analysis.

ML findings should be appended after the pending model work is complete.

Team Contributions

Team member	Main area
Ivan Chabanov	data extraction, EDA queries, pipeline integration
Alexander Mikhailov	PostgreSQL, Sqoop, Hive setup, dashboard
Danil Eramasov	documentation, storytelling, presentation materials
Bulat Fakhrutdinov	pending ML implementation and evaluation

The contribution table should be synchronized with the final report after the ML section is completed.

Conclusion

The project demonstrates a scalable chess analytics pipeline:

- raw game archives become a distributed, queryable warehouse
- board states are enriched with chess-specific features
- EDA provides interpretable signals for ratings, openings, time controls, and material balance
- the presentation is ready for the data-engineering and analytics parts
- the ML section is prepared as a clean template for the final Spark results

Thank you for your time!

References

- Chess.com public archives API
- Apache Sqoop, HDFS, Hive, Spark MLlib, Apache Superset
- Project report: `docs/report.tex`